

Отчет по решению задачи прогнозирования продаж (Favorita)

1. Постановка задачи

Целью работы является прогнозирование ежедневных продаж для каждой комбинации магазин-товар в сети Favorita на основе исторических данных и внешних факторов. Задача является соревнованием на Kaggle, метрика оценки — RMSLE (Root Mean Squared Logarithmic Error).

2. Протокол валидации и метрика

Для прогнозирования использовался временной срез с 01.01.2017 до конца данных (15.08.2017):

- Валидационная выборка: последние 16 дней (горизонт прогноза).
- Обучающая выборка: все данные до валидации.

Для анализа были выбраны 20000 самых длинных рядов. Размеры выборок после отбора:

- Train: 4 220 000 записей
- Validation: 320 000 записей

Метрика — RMSLE была выбрана, так как именно она указана в соревновании на Kaggle.

Главное отличие от обычного RMSE — мы считаем ошибку в логарифмической шкале.

В реальных данных большинство товаров продаются в небольших количествах, а некоторые редко и помногу. Если мы будем использовать обычный RMSE, огромные ошибки на редких, но крупных продажах перекроют ошибки на множестве мелких. RMSLE уравнивает вклад маленьких и больших значений в общую ошибку, то есть оценивает точность прогноза относительно масштаба продаж.

Кроме того, RMSLE штрафует занижение сильнее, чем завышение, потому что логарифм меняется быстрее при малых значениях. С точки зрения бизнеса это хорошо: недооценить спрос опаснее, чем переоценить, т.к. дефицит товара ведёт к потерянным продажам и недовольству клиентов, тогда как излишек — просто дополнительные запасы.

3. Описание данных

В работе использовались таблицы из соревнования Favorita:

- **train.csv** — история продаж (дата, магазин, товар, количество, признак промо). Целевая переменная unit_sales(количество). Большой датасет, содержащий примерно 125млн. строк.
- **items.csv** — статические характеристики товаров: семейство, класс, скоропортящийся признак.
- **stores.csv** — данные о магазинах: город, штат, тип, кластер.

- **holidays_events.csv** – праздники и события с указанием типа и локали.
- **oil.csv** – ежедневные цены на нефть.
- **transactions.csv** – количество транзакций по магазинам.

На основе этих данных сгенерированы признаки:

- Временные: год, месяц, день, день недели, выходной.
- Лаги продаж: 1, 7, 14, 28.
- Праздничные: бинарный флаг национального праздника, дни до/после.
- Цена нефти.
- Лаг транзакций.
- Категориальные идентификаторы магазина и товара, а также их статические атрибуты.

4. Описание моделей

4.1 Бейзлайны

Для установления нижней границы качества были реализованы: прогноз средним и простые статистические модели с помощью библиотеки statsforecast.

- Среднее – среднее значение по ряду (пара магазин-товар).
- Naive – наивный прогноз.
- SeasonalNaive – наивный сезонный прогноз, равен продажам 7 дней назад (недельная сезонность).
- AutoTheta – автоматический подбор модели Theta (экспоненциальное сглаживание с учётом тренда и сезонности) из библиотеки statsforecast.

4.2 CatBoost

Градиентный бустинг над деревьями решений, эффективно работающий с категориальными признаками и пропусками. Использован регрессор с параметрами:

- iterations=125
- learning_rate=0.05
- depth=6
- loss_function='RMSE'
- early_stopping_rounds=50
- Категориальные признаки переданы явно.

4.3 Temporal Fusion Transformer (TFT)

Архитектура на основе механизмов внимания, специально разработанная для временных рядов.

Параметры модели:

- hidden_size=64
- attention_head_size=4
- dropout=0.1
- hidden_continuous_size=32
- max_encoder_length=30
- max_prediction_length=16
- loss=QuantileLoss()

Обучение с оптимизатором Adam, learning rate 0.001, batch size 1024

5. Методология эксперимента

В связи с тем, что основной датасет оказался очень большим по размеру, что не позволило его даже загрузить полностью, опытным путем была подобрана дата, с которой начинается отфильтрованный датасет, позволяющая стабильно работать в Google Colab с бейзлайнами и ML моделью. Работу с трансформером пришлось перенести на Kaggle, так ресурсов Google Colab не хватило.

Также опытным путем было подобрано количество рядов, попавшее в датасет, таким образом, чтобы все модели смогли обучиться при имеющихся ресурсах и ограниченном времени.

Эксперименты проводились последовательно для оценки вклада каждой группы признаков и сравнения моделей.

1. Валидация – временной срез: последние 16 дней данных использованы как валидация, всё что раньше, начиная с 01.01.2017 – обучение.
2. Бейзлайны – прогноз средним, наивные модели и AutoTheta из библиотеки statsforecast; оценка на валидации.
3. CatBoost – обучение на подмножестве данных в Google с GPU T4. Признаки добавлялись итеративно: сначала только временные и лаги, затем статика товаров/магазинов, затем внешние признаки. Для каждой итерации фиксировалось значение RMSLE; оценка на валидации.
4. TFT – обучение в Kaggle Notebook с GPU T4.. Из-за большого объёма обучение ограничено 10 эпохами; оценка на валидации.
5. Анализ – сравнение метрик.

6. Результаты

6.1. Бейзлайны

Модель	RMSLE (на валидации)
Прогноз средним	0.60999
Naive (пред. день)	0.7809
SeasonalNaive (лаг 7)	0.72521
AutoTheta	0.62370

Наивные модели показали достаточно высокую ошибку. AutoTheta значительно улучшил результат, приблизившись к прогнозу средним, который оказался самым лучшим из бейзлайнов.

6.2. ML-модель: CatBoost

Для более качественного прогноза была выбрана модель градиентного бустинга CatBoost. Признаки создавались последовательно, чтобы оценить вклад каждой группы.

Базовый набор признаков (без экзогенных данных):

- Признаки даты: год, месяц, день, день недели, выходной.
- Лаги продаж: 1, 7, 14, 28.
- Идентификаторы магазина и товара как категориальные.

Результаты итеративного добавления признаков:

Добавленные признаки	RMSLE (валидация)	Изменение
Базовые (без экзогенных)	0.5247	—
+ данные из таблиц items и stores	0.5245	-0.0002
+ праздники (is_national_holiday)	0.5252	+0.0007
+ цена нефти (dcoilwtico)	0.5249	+0.0004
+ транзакции (lag_1)	0.5263	+0.0014
+ дни до/после праздника	0.5246	+0.0001

Лучший результат достигнут при использовании базовых признаков + информация о товарах и магазинах. Добавление остальных экзогенных признаков не привело к улучшению, а иногда даже ухудшало качество.

6.3. DL-модель: Temporal Fusion Transformer (TFT)

В качестве DL модели была выбрана архитектура TFT, которая специально разработана для анализа временных рядов с экзогенными признаками.

Обучение 10 эпох. TFT RMSLE: 0.5422556

Номер эпохи	Обучение	Валидация
Эпоха 1/10	Train Loss: 0.2434	Val Loss: 0.2178
Эпоха 2/10	Train Loss: 0.2080	Val Loss: 0.2138
Эпоха 3/10	Train Loss: 0.2033	Val Loss: 0.2120
Эпоха 4/10	Train Loss: 0.1996	Val Loss: 0.2129
Эпоха 5/10	Train Loss: 0.1965	Val Loss: 0.2132
Эпоха 6/10	Train Loss: 0.1937	Val Loss: 0.2122
Эпоха 7/10	Train Loss: 0.1912	Val Loss: 0.2134
Эпоха 8/10	Train Loss: 0.1890	Val Loss: 0.2142
Эпоха 9/10	Train Loss: 0.1870	Val Loss: 0.2153
Эпоха 10/10	Train Loss: 0.1852	Val Loss: 0.2159

Из логов видно, что модель TFT начала переобучаться после 3-й эпохи: тренировочный лосс продолжает падать, а валидационный перестал улучшаться и даже пошёл вверх. Лучший результат на валидации достигнут на 3-й эпохе. Для улучшения нужно добавить early stopping и, возможно, увеличить dropout.

Обучение 3 эпохи. TFT RMSLE: 0.5330416

7. Анализ результатов

- **Бейзлайны** показали, что даже самый простой прогноз средним даёт приемлемый результат, что значительно лучше наивных подходов.
- **CatBoost** существенно улучшил качество. Это подтверждает эффективность учёта множества факторов и нелинейных зависимостей.
- Вклад внешних признаков оказался незначительным. Наилучший прирост дали статические характеристики товаров и магазинов. Признаки праздников, нефти и транзакций не только не улучшили метрику, но и в некоторых случаях ухудшили её. Возможные причины:
 - Коллинеарность с уже имеющимися признаками (например, день недели уже отражает выходные и праздничные эффекты).
 - Модель CatBoost достаточно мощная, чтобы сама выявить нужные закономерности, и лишние признаки могут вызывать переобучение.
- **Temporal Fusion Transformer**, несмотря на более сложную архитектуру, уступил градиентному бустингу, что может быть связано с недостаточной настройкой гиперпараметров. Возможно, после донастройки качество улучшится, но если сравнивать время обучения между этими двумя моделями, то **Catboost** значительно выигрывает.